- An API gateway provides several benefits in a distributed architecture, including decoupling the internal services from the external API and offering a central, convenient place for handling cross-cutting concerns like security, monitoring, and resilience.
- Spring Cloud Gateway is based on the Spring reactive stack. It provides an API gateway implementation, and it integrates with the other Spring projects to add cross-cutting concerns to the application, including Spring Security, Spring Cloud Circuit Breaker, and Spring Session.
- Routes are the core of Spring Cloud Gateway. They are identified by a unique ID, a collection of predicates determining whether to follow the route, a URI for forwarding the request if the predicates allow, and a collection of filters that are applied before or after forwarding the request downstream.
- The `Retry` filter is for configuring retry attempts for specific routes.
- The `RequestRateLimiter` filter, integrated with Spring Data Redis Reactive, limits the number of requests that can be accepted within a specific time window.
- The `CircuitBreaker` filter, based on Spring Cloud Circuit Breaker and Resilience4J, defines circuit breakers, time limiters, and fallbacks to specific routes.
- Cloud native applications should be stateless. Data services should be used for storing the state. For example, PostgreSQL is used for persistence storage and Redis for cache and session data.
- A Kubernetes Ingress resource allows you to manage external access to applications running inside the Kubernetes cluster.